LEADREACH AI IFSC CORP.

TECHNICAL ARCHITECTURE - SERIES 2.0

# 8-Agent Asynchronous Workflow & Dependency Plan

Agent Architecture, Execution Model & Fault Tolerance

| DOCUMENT ID | CLASSIFICATION |
|---|---|
| LR-ARCH-2.2-2026 | Confidential |

| ENTITY | DATE |
|---|---|
| LeadReach AI IFSC Corp., Ontario Corporation | March 2026 |

| LLM PROVIDER | AGENT COUNT |
|---|---|
| Zhipu AI GLM | 8 Specialized AI Agents |

GLM-4.6V-FLASH | GLM-4.7-FLASH | PGVECTOR | PRISMA ORM | 17+ CHANNELS

SECTION 1

# Architecture Overview

LeadReach deploys eight specialized AI agents that operate asynchronously through an event-driven orchestration system. The architecture follows a supervisor pattern where the Orchestrator agent decomposes high-level requests into execution plans, delegates tasks to specialized agents, and synthesizes results. Each agent maintains independent state, memory, and channel assignments while communicating through a shared database layer (Prisma ORM + Supabase PostgreSQL with pgvector).

| Agent | Primary Role | Stage |
|---|---|---|
| Orchestrator | Workflow decomposition, delegation & synthesis | 1 |
| Prospect Discovery | Company & prospect identification across channels | 2 |
| Data Enrichment | Lead enrichment with multi-source data | 3a |
| Web Research | Market intelligence & competitive research | 3b |
| Lead Qualification | BANT / MEDDIC / ICP scoring | 4 |
| Outreach Composer | Multi-channel outreach content generation | 5 |
| Pipeline Manager | Stage transitions & follow-up scheduling | 6 |
| Report Generator | Campaign reporting & AI-driven insights | 7 |

The 10-module infrastructure layer (Sessions, Models, Logs, Cron, Skills, Plugins, Profiles, Config, Keys, Documentation) provides cross-cutting services. LLM inference is powered by Zhipu AI GLM (glm-4.6v-flash primary, glm-4.7-flash fallback). The Agent-Reach Bridge connects 17+ outbound channels with multi-tier fallback routing.

SECTION 2

# Agent Dependency Graph

The dependency graph defines the execution order and data-flow relationships between agents. Stages 3a and 3b execute in parallel; stages 2 -> 3a -> 4 -> 5 -> 6 are sequential.

| Stage | Agent | Depends On | Produces | Consumes |
|---|---|---|---|---|
| 1 - Orchestration | Orchestrator | User Request | Execution Plan | Agent Results |

| | | | | |
|---|---|---|---|---|
| 2 - Discovery | Prospect Discovery | Execution Plan | Raw Prospects | Search Queries, ICP Criteria |
| 3a - Enrichment | Data Enrichment | Raw Prospects | Enriched Leads | Company URLs, LinkedIn Profiles |
| 3b - Research | Web Research | Execution Plan | Market Intelligence | Company Names, Industry Data |
| 4 - Qualification | Lead Qualification | Enriched Leads | Scored Leads | BANT/MEDDIC/ICP Frameworks |
| 5 - Outreach | Outreach Composer | Scored Leads | Outreach Sequences | Templates, ICP Profiles |
| 6 - Pipeline | Pipeline Manager | Outreach Sequences | Pipeline Updates | Stage Rules, Follow-up Schedules |
| 7 - Reporting | Report Generator | All Agent Outputs | Reports & Insights | Pipeline Data, Score Distributions |

**Key:** Stages 3a and 3b execute in PARALLEL. Stages 2 -> 3a -> 4 -> 5 -> 6 are SEQUENTIAL.

## SECTION 3

# Asynchronous Execution Model

The platform uses a task-queue pattern where the Orchestrator creates AgentTask records in the database. Each agent polls for pending tasks matching its skill set, executes them idempotently, and writes results back as JSON. The Orchestrator monitors completion and triggers dependent stages automatically.

- The Orchestrator creates AgentTask records in the database
- Each agent polls for pending tasks matching its skill set
- Task execution is idempotent (safe to retry)
- Results are written back to AgentTask.output as JSON
- The Orchestrator monitors completion and triggers dependent stages
- Long-running tasks (deep search, multi-channel enrichment) execute with progress tracking

## SECTION 4

# Agent Communication Protocol

Agents communicate through five distinct mechanisms, each optimized for latency and use case. The communication layer balances real-time responsiveness with reliable asynchronous processing.

| Communication Type | Mechanism | Latency | Use Case |
|---|---|---|---|
| Synchronous Request | Direct function call | < 10 ms | Simple queries, health checks |
| Asynchronous Task | Database queue | 100 ms - 5 min | Pipeline stages, multi-step workflows |
| Event Notification | Pub/Sub (in-process) | < 50 ms | Status updates, completion signals |
| Shared State | Prisma ORM (PostgreSQL) | 5 - 20 ms | Agent session data, lead records |
| Memory Retrieval | pgvector similarity search | 10 - 50 ms | Context enrichment, experience recall |

SECTION 5

# Agent Skill Registry

Each agent exposes a set of typed skills with defined input/output schemas and measured latency. The skill registry enables the Orchestrator to route tasks to the correct agent and skill combination.

| Agent | Skill | Input Schema | Output Schema | Avg Latency |
|---|---|---|---|---|
| Orchestrator | plan_workflow | Request text | Execution plan JSON | 2.1 s |
| Prospect Discovery | discover_companies | ICP criteria, channels | Company list JSON | 8.4 s |
| Prospect Discovery | deep_search | Company name, sub-queries | Deep profile JSON | 12.6 s |
| Data Enrichment | enrich_leads | Lead IDs, channels | Enrichment data JSON | 6.2 s |
| Data Enrichment | extract_contacts | Lead IDs, sources | Contact list JSON | 4.8 s |
| Web Research | research_company | Company name, depth | Research report JSON | 10.3 s |
| Web Research | market_analysis | Industry, competitors | Analysis report JSON | 14.7 s |
| Lead Qualification | score_bant | Lead data | BANT score JSON | 1.8 s |

| | | | | |
|---|---|---|---|---|
| Lead Qualification | score_meddic | Lead data | MEDDIC score JSON | 2.4 s |
| Lead Qualification | score_icp | Lead data, ICP profile | ICP score JSON | 2.0 s |
| Outreach Composer | compose_email | Lead data, framework | Email draft JSON | 3.6 s |
| Outreach Composer | compose_linkedin | Lead data, framework | LinkedIn message JSON | 3.2 s |
| Outreach Composer | design_sequence | Lead data, sequence type | Sequence JSON | 5.4 s |
| Pipeline Manager | manage_stages | Pipeline event | Stage update JSON | 0.8 s |
| Pipeline Manager | schedule_followups | Lead data, rules | Follow-up schedule JSON | 1.2 s |
| Report Generator | pipeline_report | Campaign ID | Report JSON | 4.8 s |
| Report Generator | ai_insights | Report data | Insights JSON | 6.2 s |
| Report Generator | action_items | Report data | Action items JSON | 3.4 s |

SECTION 6

# Three-Tier Memory Architecture

LeadReach implements a three-tier memory system inspired by human cognition, enabling agents to learn from past interactions, store domain knowledge, and optimize workflows over time.

| Memory Type | Storage | Retrieval | Retention | Purpose |
|---|---|---|---|---|
| Episodic | AgentSession.embeddings | pgvector cosine similarity | 90 days | Past interactions, campaign experiences |
| Semantic | AgentSession.context | Structured JSON query | Indefinite | Learned insights, domain knowledge |
| Procedural | AgentTask.output patterns | Skill registry lookup | Indefinite | Workflow optimization, success rates |

Episodic Memory stores vectorized interaction embeddings, enabling agents to recall similar past campaigns and apply learned patterns via pgvector cosine similarity search.

Semantic Memory maintains structured domain knowledge - ICP profiles, industry taxonomies, and learned rules - persisted indefinitely in the session context layer.

Procedural Memory captures workflow success patterns in task output, allowing the skill registry to optimize routing and reduce latency over time.

SECTION 7

# Fault Tolerance & Recovery

The architecture is designed for resilient operation with multiple fallback mechanisms, ensuring the pipeline never stalls and never returns zero results.

Circuit breaker pattern: agents in error state are temporarily disabled to prevent cascade failures

Retry with exponential backoff: 3 attempts per task, 3s x attempt number

Dead letter queue: failed tasks after max retries are logged for manual review

Graceful degradation: if Discovery fails, LLM knowledge generation provides fallback leads

Zero-result guarantee: the pipeline NEVER returns zero prospects under any condition

Health monitoring: each agent reports status via AgentReachChannel model

Automatic recovery: agents self-heal after transient errors without operator intervention

SECTION 8

# Scalability Design

The agent architecture is designed to scale horizontally while maintaining cost discipline. Stateless workers, efficient indexing, and multi-model routing ensure performance at any volume.

| Mechanism | Description | Impact |
|---|---|---|
| Horizontal Scaling | Agents are stateless workers; multiple instances can run concurrently | Linear throughput increase |
| Database Sharding | pgvector indexes enable efficient similarity search at scale | Sub-50ms recall at 1M+ vectors |
| Rate Limiting | Per-agent rate limits prevent API quota exhaustion | Zero dropped tasks from quota errors |
| Cost Control | Thinking budget allocation prevents token waste | Predictable per-lead cost |
| Multi-Model Routing | Primary/fallback model selection optimizes cost/performance | 15-30% cost reduction vs. single-model |

The combination of these mechanisms ensures that LeadReach can handle enterprise-scale outbound campaigns - from single-digit daily leads to tens of thousands - without architectural changes.